TEK-UP University

ING-3

Module 1 : Programmation C

Chapitre 1 : Les bases de la programmation en C

Élaboré par : Ingénieure Nahla AROUA

Plan

1. Présentation du langage
2. Installation de l’environnement CodeBlocks
3. Types simples
4. Actions simples

1. Présentation du langage

Élaboré par : Ingénieure Nahla AROUA

Langage C

C’est un des langages de programmation les plus connus et les plus utilisés.

Langage compilé

Un langage où le code source est converti en code binaire exécutable par la machine grâce à un programme nommé **compilateur**.

Langage de bas niveau

Un langage de programmation plus proche du code machine

Programmation procédurale

C’est un langage basé sur les procédures.

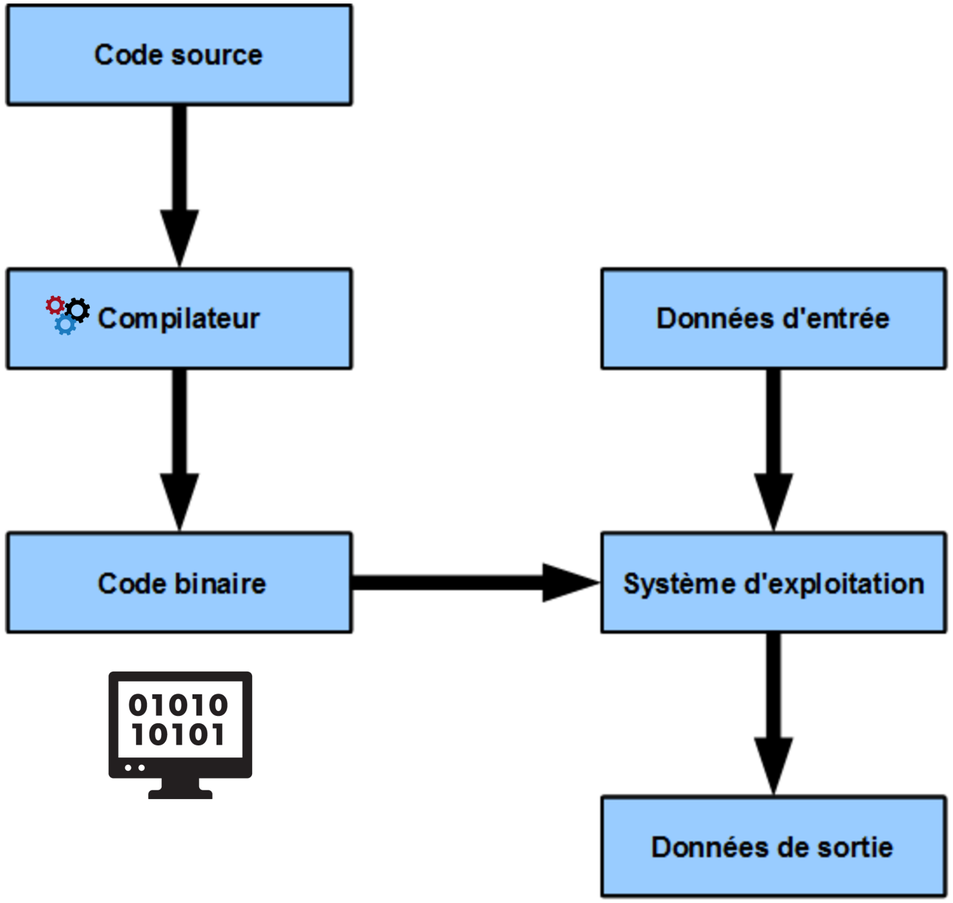
Une **procédure** est une séquence d’instructions regroupées sous un même nom.

Gestion de la mémoire

Un processus manuel d'**allocation** et de **libération de mémoire** au moment de l'exécution.

Processus de compilation

```c
#include <stdio.h>

int main() //le main
{
    /* La fonction printf() est utilisée pour :
    afficher du texte sur la console.
    */
    printf("Hello World !\n");

    // Le programme se termine avec succès en retournant 0.
    return 0;
}
```



Avantages du langage C

Portabilité

Le langage peut être utilisé sur différents systèmes d'exploitation ou matériels sans nécessité de modifications importantes du code source.

Contrôle fin sur le processeur

Les programmeurs ont un accès direct à la mémoire du système.

Avantages du langage C

Efficacité

Le langage est capable de produire un code performant..

Rapidité

La réponse et l'exécution des opérations demandées sont rapides.

Applications du langage C

01 Systèmes d'exploitation

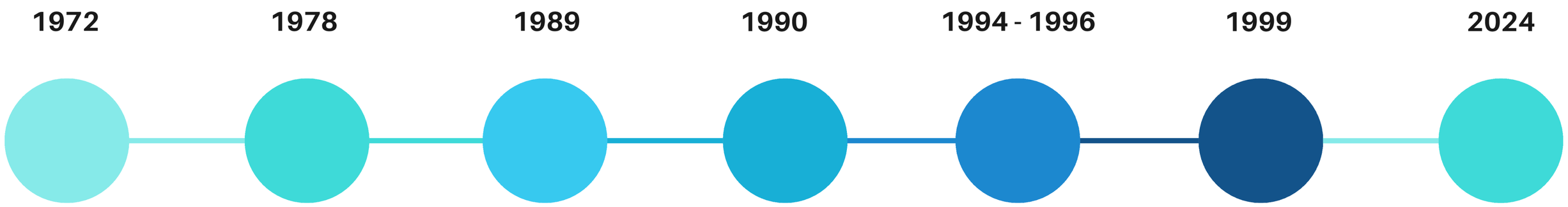
02 Systèmes embarqués et IoT

03 Jeux vidéo

04 Programmes d'exécution

HISTOIRE

Le langage C a été développé par Ken Thompson et Dennis Ritchie dans les Laboratoires Bell en 1972, dans le cadre du développement du système UNIX.



Structure du programme C

Un programme écrit en langage C est composé de deux parties :

01

Déclarations

Cette partie contient les déclarations de :

1. Bibliothèques.
2. Constantes
3. Variables globales.
4. Prototypes de fonctions.

Structure du programme C

02

Définitions des fonctions

Fonction main() :

- C’est la principale fonction qui est utilisée par le système pour exécuter le programme.

- Elle représente le corps du programme.

Structure du programme C

02

Définitions des fonctions

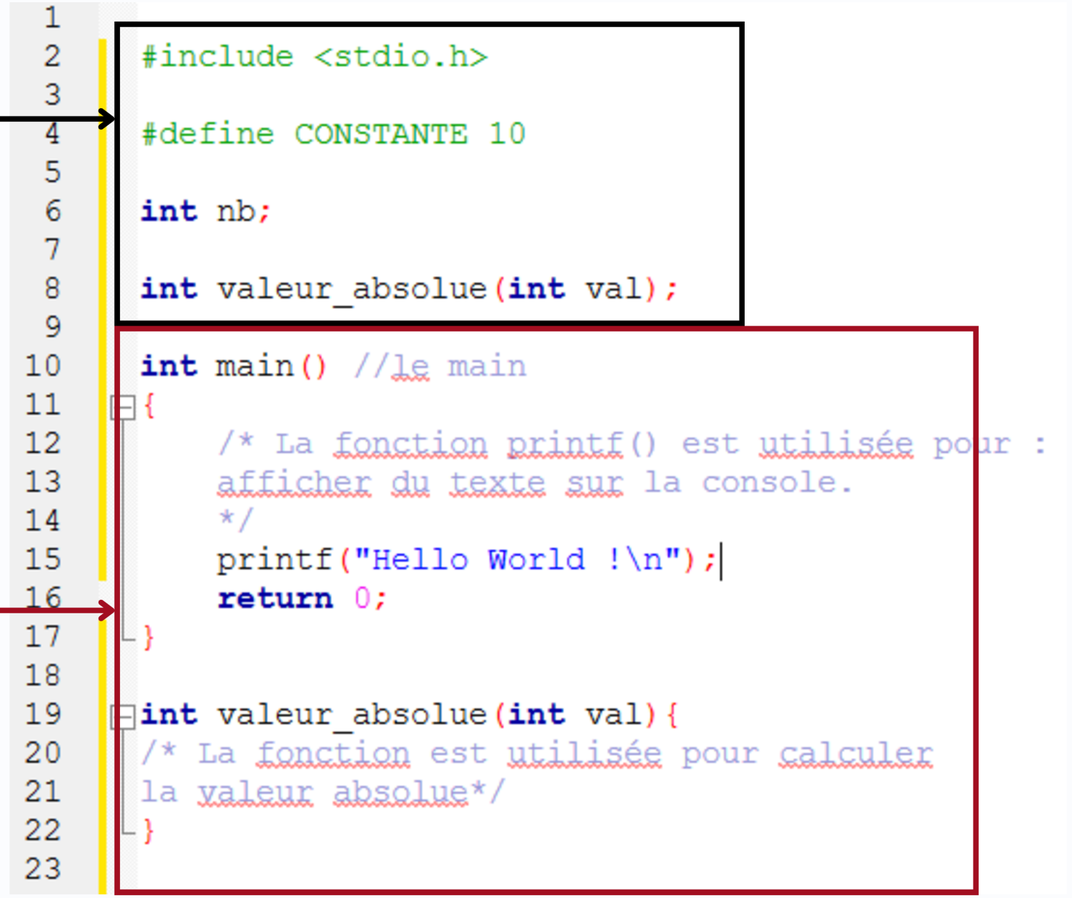
Fonctions personnalisées

- Elles peuvent être appelées depuis la fonction main() ou bien depuis d'autres fonctions. Ce qui rend le code modulaire et simple à gérer.

Structure du programme C

Déclarations

Définitions des fonctions



```c
#include <stdio.h>

#define CONSTANTE 10

int nb;

int valeur_absolue(int val);

int main() //le main
{
    /* La fonction printf() est utilisée pour :
    afficher du texte sur la console.
    */
    printf("Hello World !\n");
    return 0;
}

int valeur_absolue(int val){
/* La fonction est utilisée pour calculer
la valeur absolue*/
}
```

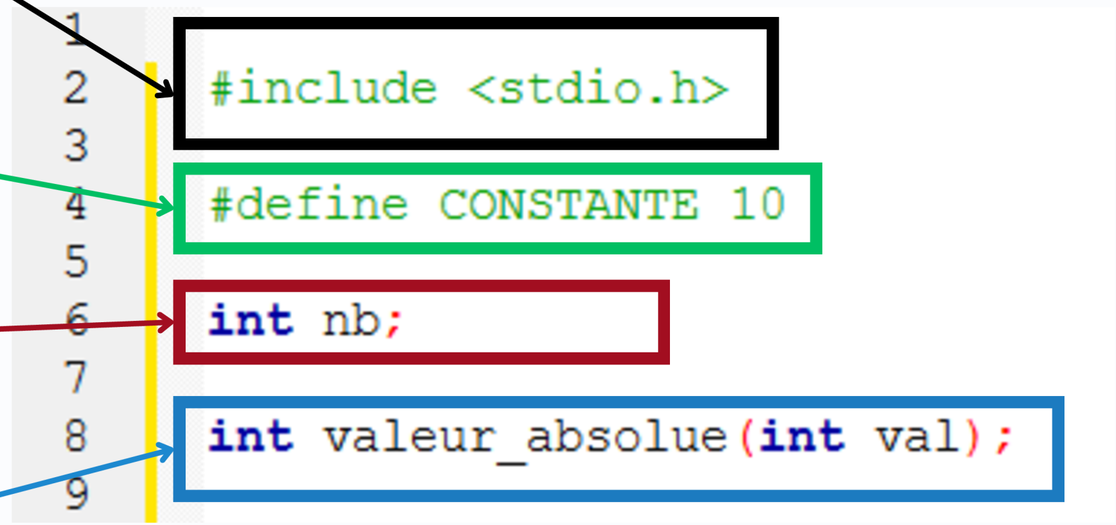
Structure du programme C

Déclaration de directive du préprocesseur

Déclaraction d'une constante

Définitions d'une variable

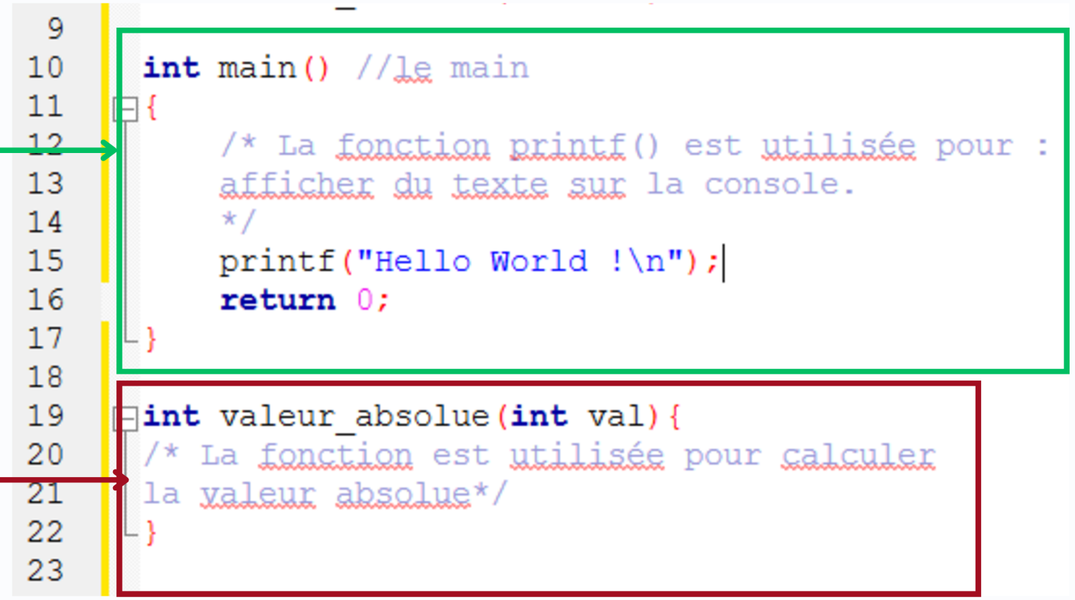
Déclaration d'un prototype de fonction.



Structure du programme C

Fonction main()

Fonction valeur_absolue()



```
 9
10  int main() //le main
11  {
12      /* La fonction printf() est utilisée pour :
13      afficher du texte sur la console.
14      */
15      printf("Hello World !\n");
16      return 0;
17  }
18
19  int valeur_absolue(int val){
20  /* La fonction est utilisée pour calculer
21  la valeur absolue*/
22  }
23
```

Régles et conventions

- Le langage C est sensible à la casse (en anglais case sensitive), cela signifie qu'un nom contenant des majuscules est différent du même nom écrit en minuscules.

- **Par exemple :** main() non Main() **et** printf() non Printf().

Régles et conventions

- Toute instruction se termine par un point-virgule.
- **Par exemple :** printf(“Hello World !”);

Régles et conventions

- C possède une bibliothèque standard (souvent appelée libc) comportant les fonctions pour des tâches courantes telles que l'entrée/sortie et la gestion de fichiers. Ce qui permet un gain de temps.

- **Par exemple :** le fichier <stdio.h> (Standard Input/Output) : contient des fonctions pour les E/S de base sur les fichiers et la console comme printf().

Régles et conventions

Les règles de nommage des variables en C :

Le nom de variable en C doit respecter les conditions suivantes :

- Il doit commencer par une lettre (majuscule ou minuscule) ou un tiret de soulignement (_).
- Il peut contenir des lettres, des chiffres ou des tirets de soulignement.
- Il est sensible à la casse (par exemple, age et Age sont deux variables différentes).
- Il ne peut pas être un mot-clé réservé du langage C (comme int, for, while, if, etc.).

Conventions de nommage

Conventions de nommage	Exemples
Pascal Case	DateDeNaissance
Camel Case	dateDeNaissance
Snake Case	date_de_naissance
Cobol Case	DATE-DE-NAISSANCE
Pascal Snake Case	Date_De_Naissance

Conventions de nommage

Conventions de nommage	Exemples
Camel Snake Case	date-De-Naissance
Kebab Case	date-de-naissance
HTTP Header Case	Date-De-Naissance
Screaming Snake Case	DATE_DE_NAISSANCE
Flat Case	datedenaissance
Upperflat Case	DATEDENAISSANCE

Commentaire

- **Commentaire** est une ligne ou un bloc de texte qui permet de **rendre** le code plus lisible et plus maintenable.

Remarque :

- Le compilateur ne prend pas en compte les commentaires.

Commentaire

01

Commentaire en ligne

// ce commentaire

02

Commentaire multiligne = bloc de texte

/* Commentaire multiligne =
bloc de texte
*/

Exemple :

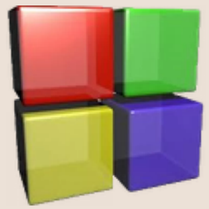
```c
#include <stdio.h>

int main() //le main
{
    /* La fonction printf() est utilisée pour :
    afficher du texte sur la console.
    */
    printf("Hello World !\n");

    // Le programme se termine avec succès en retournant 0.
    return 0;
}
```

2. Installation de l’environnement CodeBlocks

Élaboré par : Ingénieure Nahla AROUA

Editeurs de code

Un éditeur de code est un logiciel de traitement de texte spécialisé, conçu pour écrire, modifier et organiser le code source des programmes informatiques.
Pour coder en langage C, les éditeurs de code sont les suivants :



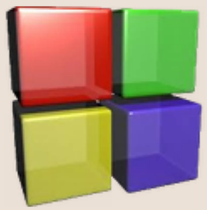
Code::Blocks

VS Code

NetBeans

Eclipse

Code::Blocks

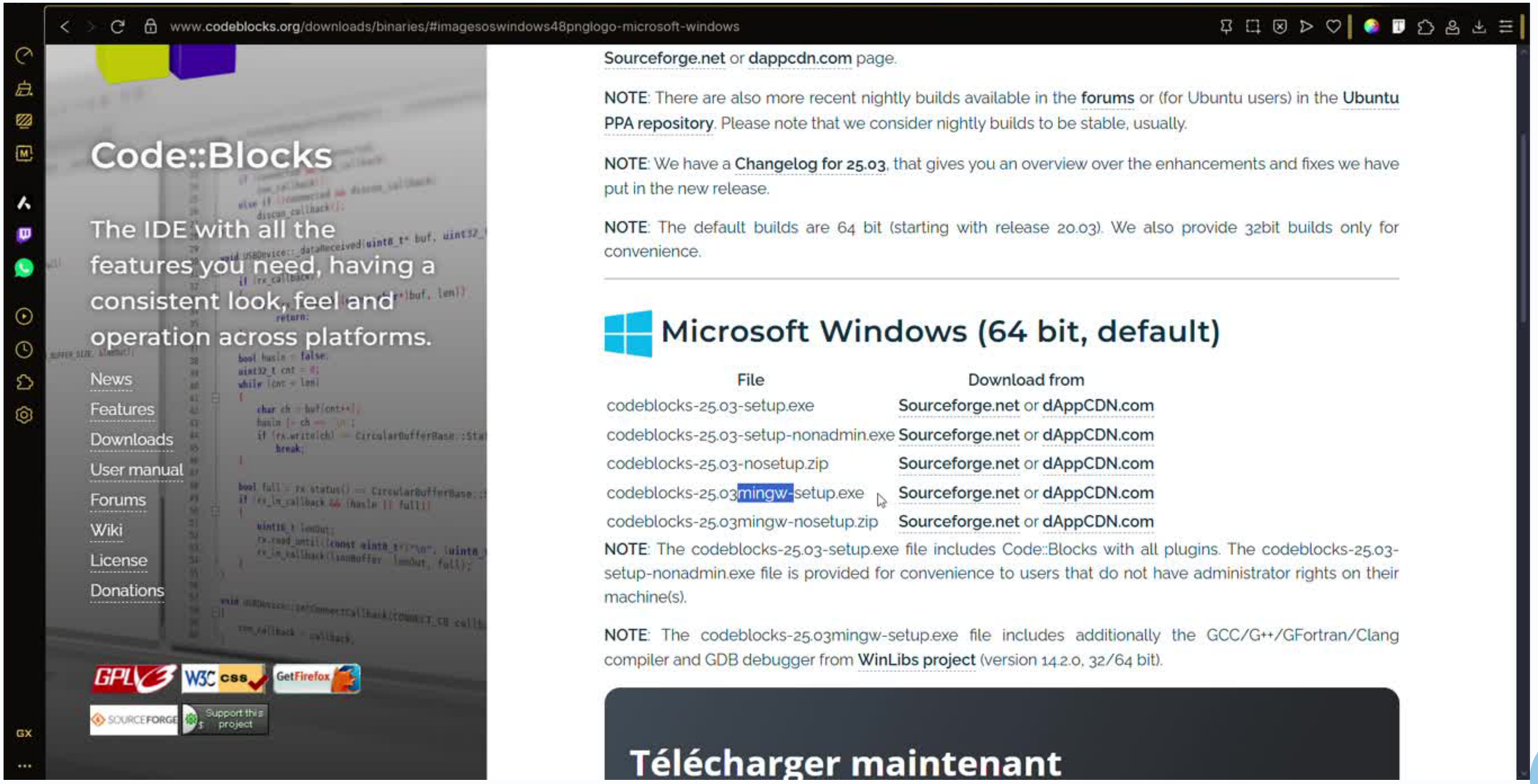


Code::Blocks

- **Code::Blocks** est un environnement de développement intégré (IDE) gratuit que nous allons utiliser durant nos cours.

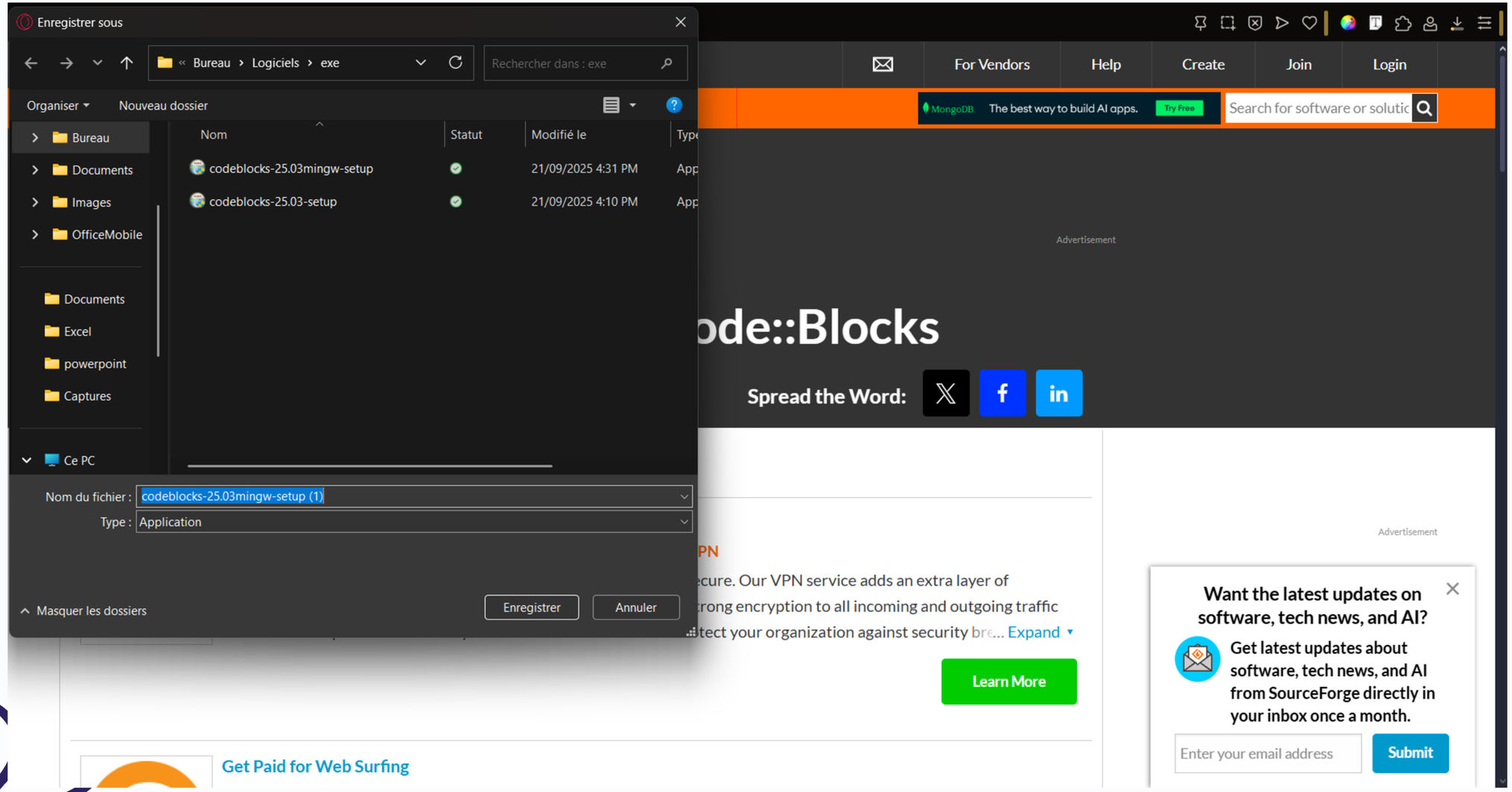
- Cet IDE utilise la bibliothèque wxWidgets. Il est utilisé pour la compilation du langage C, C++ et Fortran .

Téléchargement de Code::Blocks

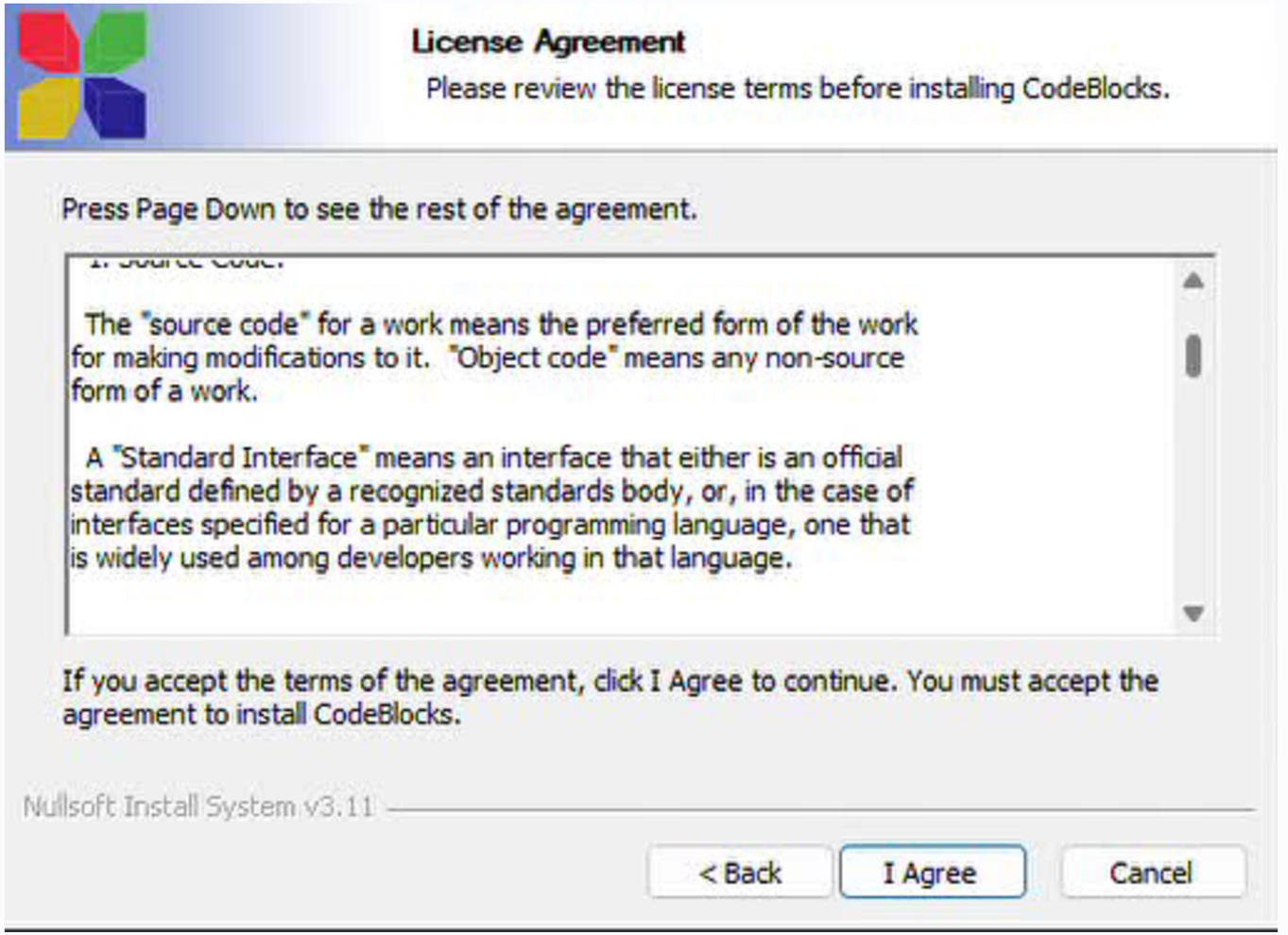


Téléchargement de Code::Blocks

Enregistrer le fichier .exe.



Installation de Code::Blocks



Remarque

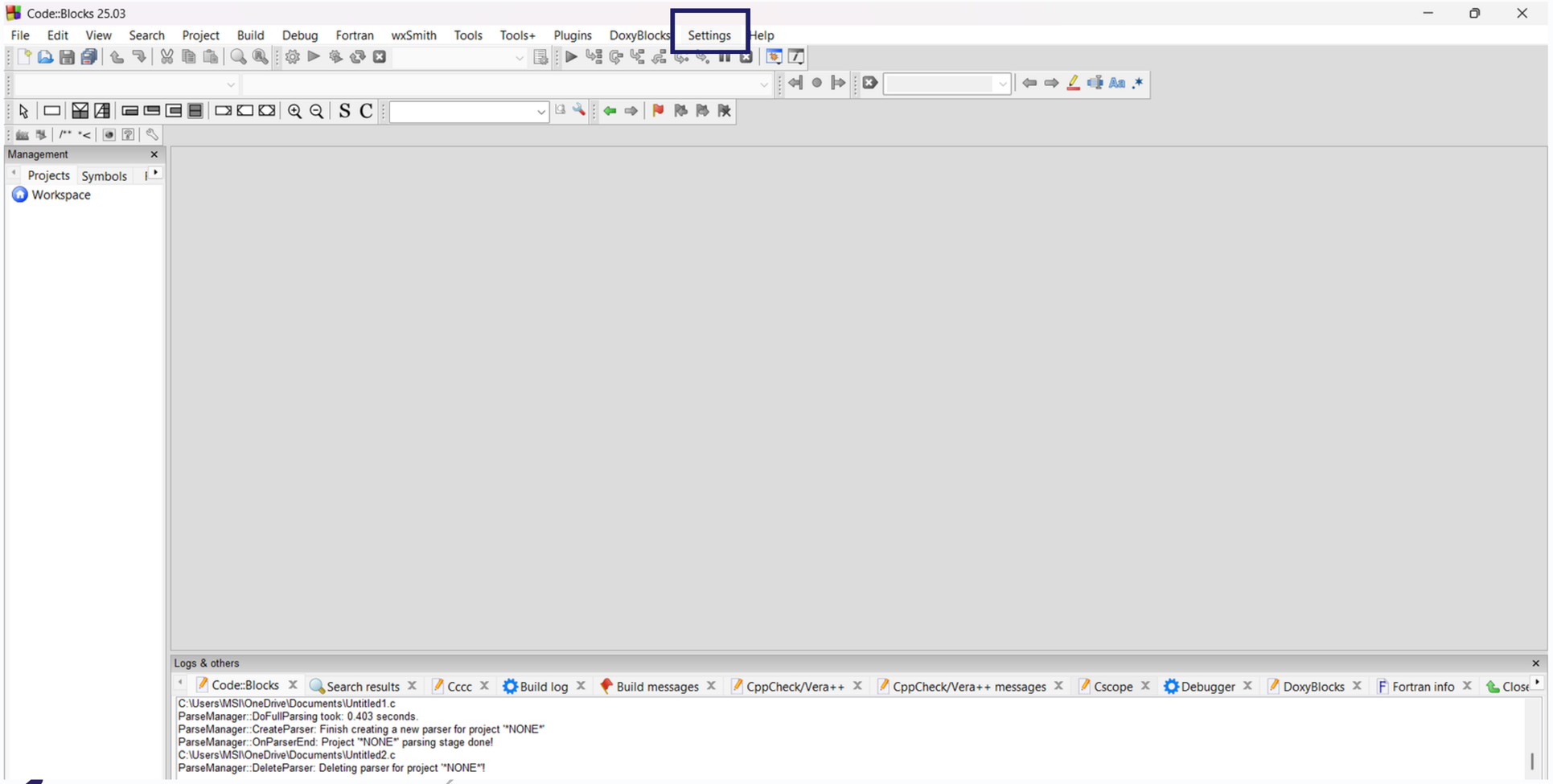
- **Si la compilation ne fonctionne pas** et vous avez l'erreur suivante :

"Info[Environment error]: Can't find compiler executable in your configured search paths for Intel C/C++ Compiler",
cela signifie que vous avez téléchargé la version de Code::Blocks sans **mingw** (le compilateur).

- Retournez sur le site de Code::Blocks et téléchargez la version avec **mingw**.

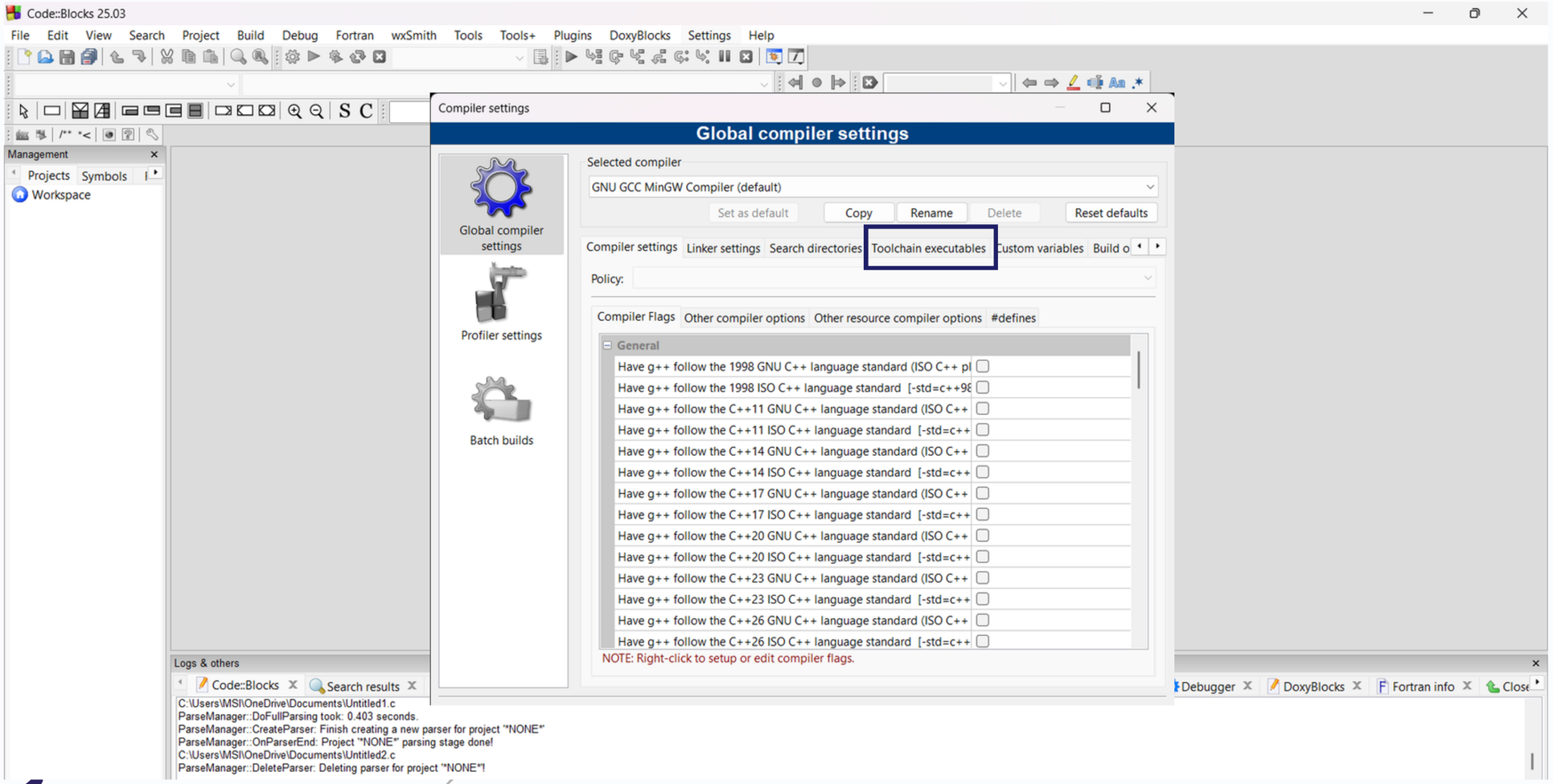
Configuration du compilateur

1. Sélectionnez **settings.**
2. Sélectionnez **Compiler.**



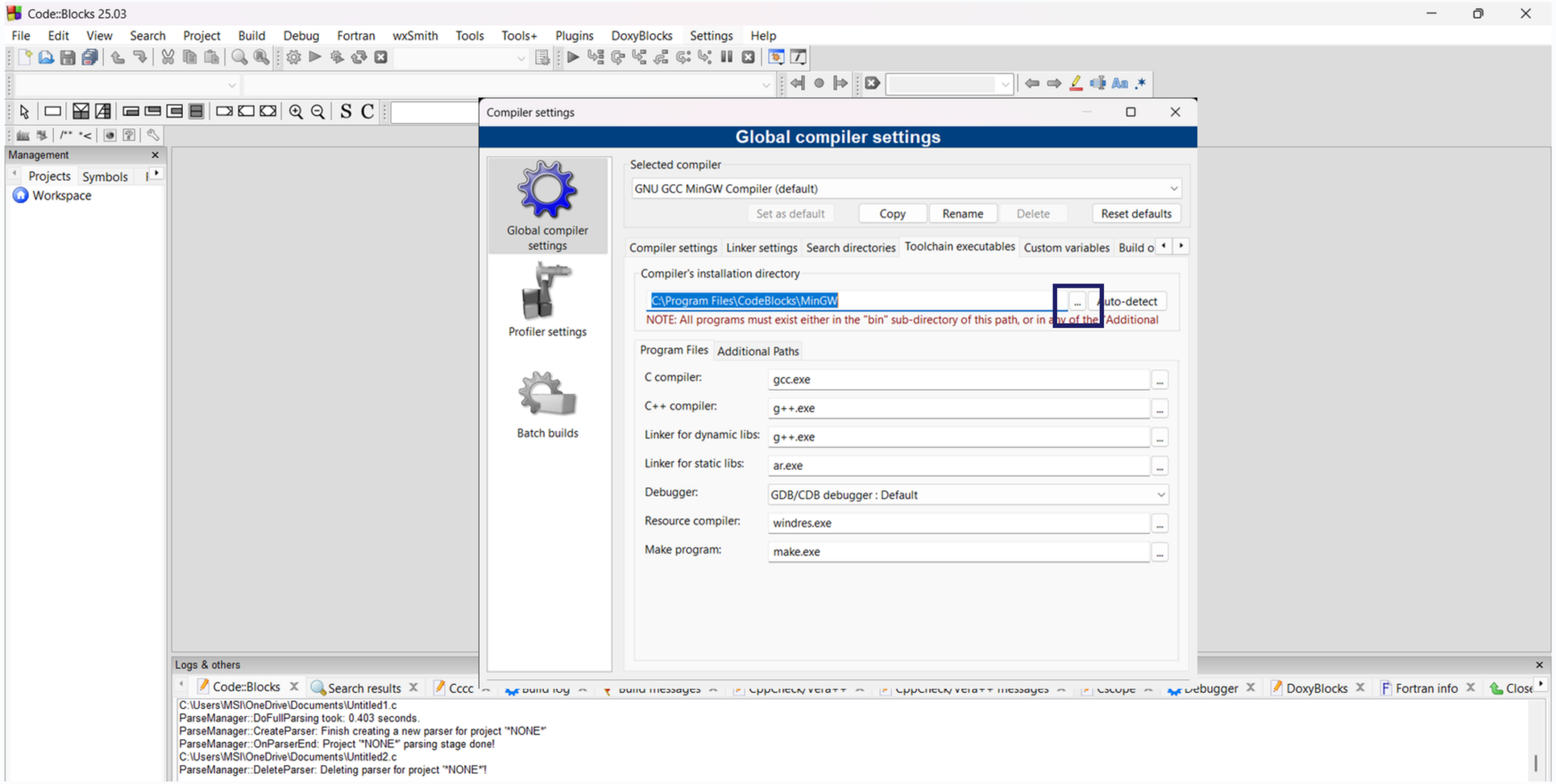
Configuration du compilateur

3. Sélectionnez **Toolchain exacutables.**



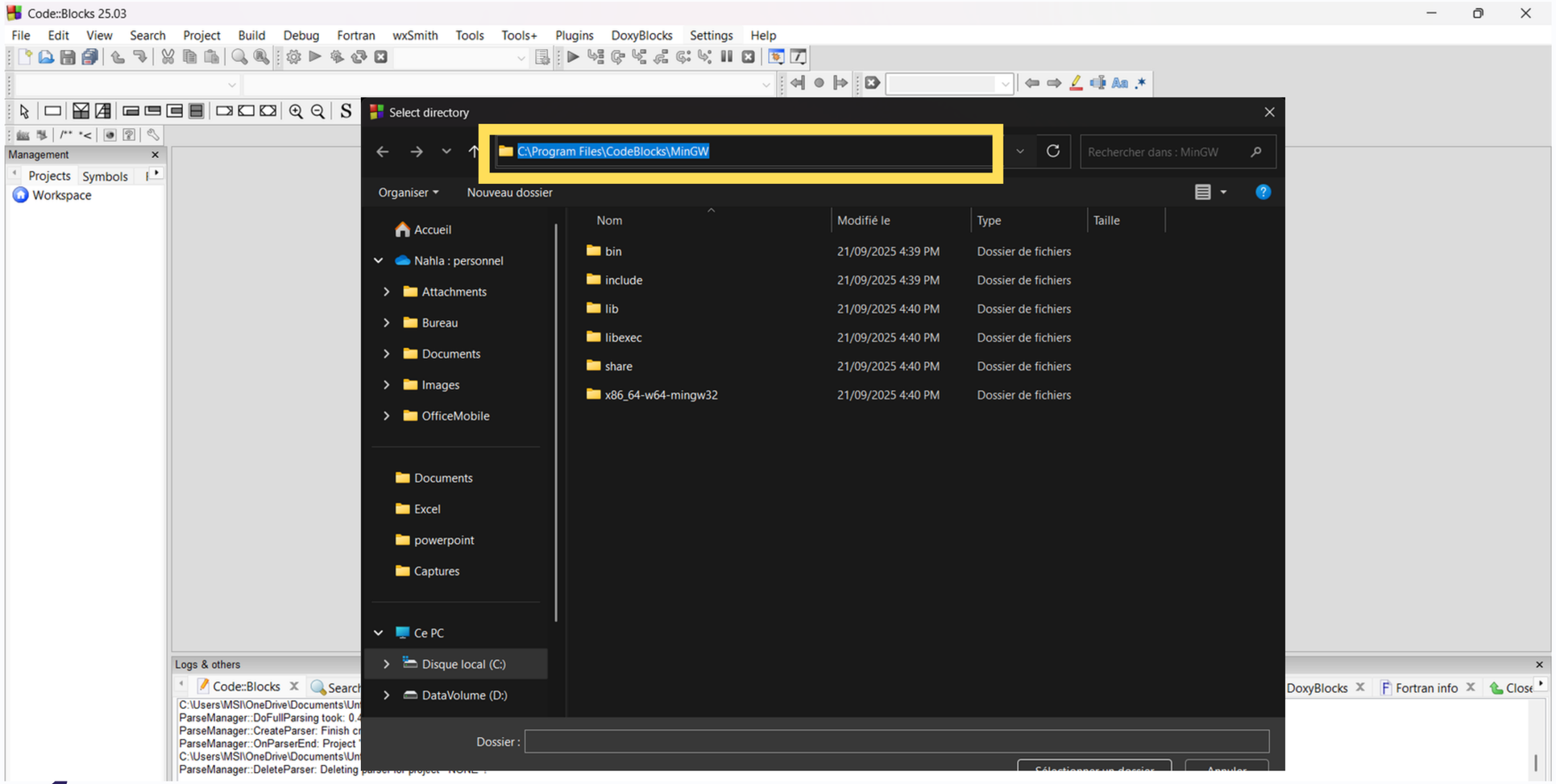
Configuration du compilateur

4. Sélectionnez **les trois points à droite.**



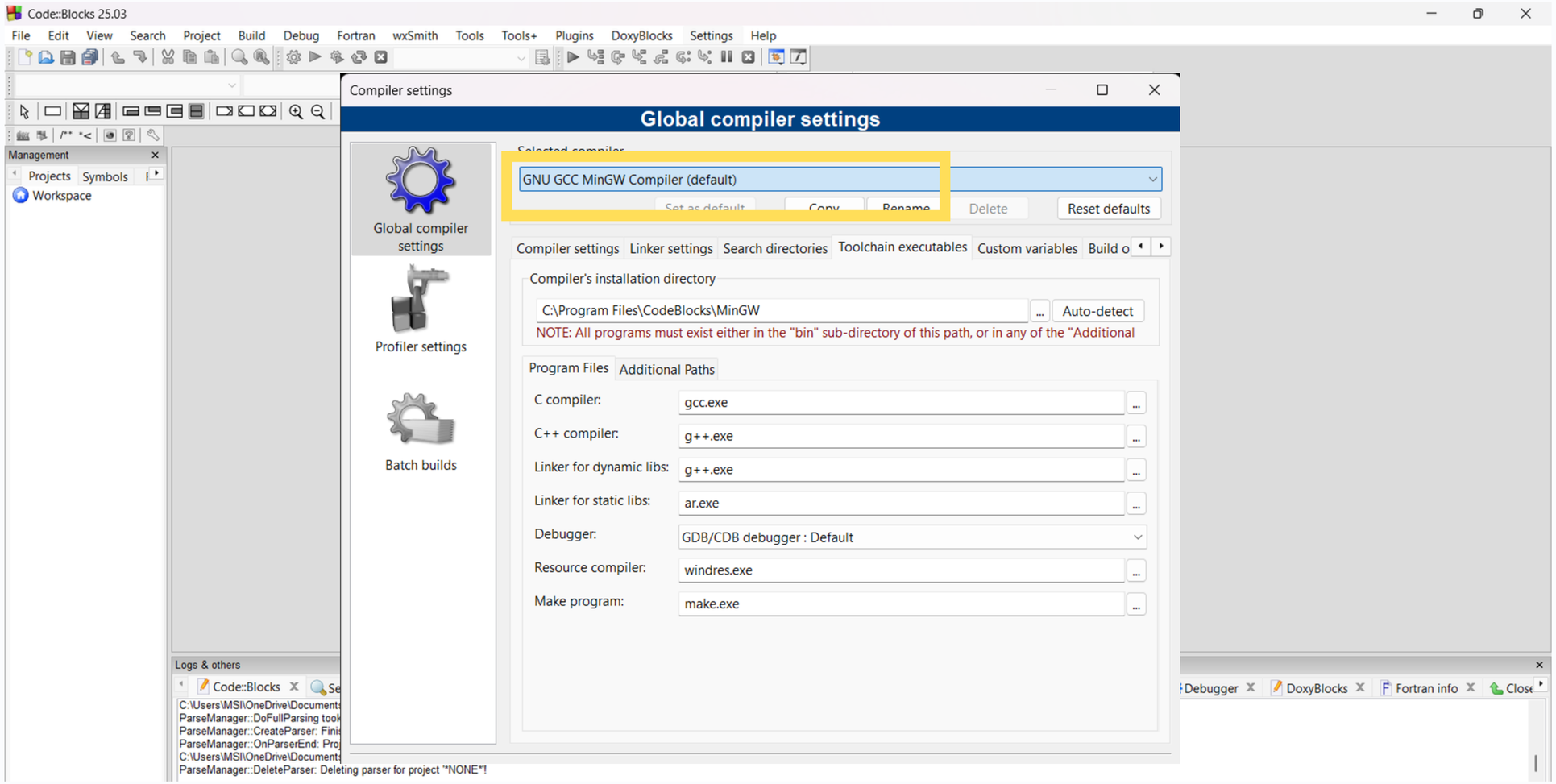
Configuration du compilateur

5. Accédez au chemin **C:\Program Files\CodeBlocks\MinGW.**
6. Sélectionnez le dossier.



Configuration du compilateur

7. Sélectionnez **le compilateur.**



3. Types simples

Élaboré par : Ingénieure Nahla AROUA

Variables et constantes

Élaboré par : Ingénieure Nahla AROUA

Variable

- Une variable correspond à **une portion de mémoire**, à laquelle nous attribuerons un nom. Ce nom permettra d'identifier notre variable.

- Nous pouvons nommer les données que nous **manipulons**, chacun de ces noms étant remplacé lors de la **compilation par une adresse**.

Constante

une constante est une valeur qui **reste fixe** durant l'exécution d'un programme et qui ne peut être modifiée sauf au niveau de leurs déclarations.

Constante

La directive du préprocesseur #define remplace toutes les occurrences de la constante par une valeur spécifiée.

Par exemple : #define MAX 100

Entier (int)

Élaboré par : Ingénieure Nahla AROUA

Entier (int)

- Le type int représente un nombre entier positive ou négative sans partie décimale.
- **Déclaration :** Type_variable nom_variable = valeur;
- **Exemple : int** X= 5;

Entier (int)

Les types dérivés de type int sont les suivants :

- **int** : Le bit le plus à gauche est utilisé pour le signe (0 pour positif/zéro, 1 pour négatif).
- **unsigned int** : Tous les bits sont utilisés pour représenter la valeur.
- **short int** : Permet d'économiser de la mémoire lorsque les valeurs sont garanties petites.
- **long int** : Utilise plus de mémoire que int (souvent 8 octets sur les systèmes 64 bits) pour stocker des nombres très grands.
- **unsigned short int**
- **unsigned long int**

Réel
(float)

Élaboré par : Ingénieure Nahla AROUA

Réel (float)

- Le type float est un nombre à virgule flottante en simple précision.
- Précision décimale : Environ 6 à 7 chiffres décimaux.

- **Par exemple** : x = 3.14 est considéré comme double par défaut ; pour le spécifier comme float, on ajoute le suffixe f ou F (par exemple, 3.14f).

Réel
(double)

Élaboré par : Ingénieure Nahla AROUA

Réel (double)

- Le type double est un nombre à virgule flottante en Double précision.
- Précision décimale : Environ 15 chiffres décimaux.

Caractère (char)

Élaboré par : Ingénieure Nahla AROUA

Caractère (char)

- Le type char représente un caractère et prend toutes les valeurs du code ASCII (soit 256 valeurs de 0 à 0xFF).
- Il permet de déclarer une variable qui occupe un seul octet en mémoire.

- Le type char est par défaut non signé (valeurs entre 0 et 255) ou signé (valeurs entre -128 et +127).

Caractère (char)

- **Déclaration et initialisation** : Type_variable nom_variable = valeur;
- **Exemple** : char X= ‘5’;

Booléen (bool)

Élaboré par : Ingénieure Nahla AROUA

Booléen (bool)

- Le type bool est une variable ou constante de type booléen qui prend la valeur true ou false, en ajoutant l'entête #include<stdbool.h> au fichier c.

- La valeur booléenne est stocké comme un entier en mémoire :
  l’entier 0 pour false et tout entier non nul (généralement 1) pour true.

- Déclaration et initialisation : bool var = false;

Booléen (bool)

Utilisation :

- Tester des conditions.
- Contrôler le flux des programmes(if, while..).
- Représenter des états (échec/succès, activé/désactivé).

Booléen (bool)

```c
#include <stdbool.h>
#include <stdio.h>

int main()
{
    printf("true = %u, false = %u\n", true, false);
    return 0;
}

```

Résultat

```
true = 1, false = 0

Process returned 0 (0x0)   execution time : 0.076 s
Press any key to continue.
```

Booléen (bool)

```c
#include<stdio.h>
#include<stdbool.h>
bool A,B,C;
int main(){
    A=true;
    B=false;
    C = !A == B;
    printf("%d \n",C);
return 0;
}
```

Résultat

```
1

Process returned 0 (0x0)   execution time : 0.100 s
Press any key to continue.
```

Type	Signification	Taille (en octets)	Plage de valeurs acceptée
char	Caractère	1	-128 à 127
unsigned char	Caractère non signé	1	0 à 255
short int	Entier court	2	-32 768 à 32 767
unsigned short int	Entier court non signé	2	0 à 65 535
int	Entier	2 4	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned int	Entier non signé	2 4	0 à 65 535 0 à 4 294 967 295
long int	Entier long	4	-2 147 483 648 à 2 147 483 647
unsigned long int	Entier long non signé	4	0 à 4 294 967 295

Type	Signification	Taille (en octets)	Plage de valeurs acceptée
float	Flottant (réel)	4	$3.4*10^{-38}$ à $3.4*10^{38}$
double	Flottant double	8	$1.7*10^{-308}$ à $1.7*10^{308}$
long double	Flottant double long	10	$3.4*10^{-4932}$ à $3.4*10^{4932}$

Opérateurs arithmétiques et logiques

Élaboré par : Ingénieure Nahla AROUA

Opérateurs arithmétiques

Le tableau ci-dessous reprend les différents opérateurs arithmétiques :

+	**Addition**
-	**Soustraction**
*	**Multiplicateur**
%	**Modulo**
/	**Division**

Opérateurs arithmétiques :

Les opérateurs arithmétiques **+, -, *** et **/** s'appliquent aux types réels et entiers, ainsi que les types caractères et booléens. L'opérateur modulo % est quant à lui strictement réservé aux entiers. Dans une expression arithmétique, les booléens sont implicitement convertis en valeurs numériques (0 pour false, 1 pour true). De même, le type char est traité comme un entier sur un octet correspondant au code ASCII du caractère.

Remarque : Le langage C ne possède pas les mots-clés div et mod. La division entière est réalisée au moyen de l'opérateur **/** (lorsque les deux opérandes sont entiers), tandis que le reste de la division est obtenu à l'aide de l'opérateur **%**.

Opérateurs logiques

Le tableau ci-dessous reprend les différents opérateurs logiques :

&&	**Et logique**
||	**OR logique**
!	**Not logique**
^	**Ou exclusif**

Opérateurs logiques

Ces opérateurs peuvent être appliqués aux types réels, entiers, caractères et booléens. Et le résultat obtenu est un booléen ; 1 (true) et 0 (false) .

A	B	A && B	A || B	! A	A ^ B
true	false	0	1	0	1
true	true	1	1	0	0
false	true	0	1	1	1
false	false	0	0	1	0

Opérateurs de comparaison :

Ces opérateurs peuvent être appliqués aux types réels, entiers, caractères et booléens. Et le résultat obtenu est un booléen ; 1 (true) et 0 (false) .

3 == 3	égalité
3 != 4	inégalité
3 >= 1	supérieure ou égale
4 > 3	supérieure
3 < 4	inférieure
3 <= 4	inférieure ou égale

Opérateurs de comparaison :

```c
#include<stdio.h>
#include<stdbool.h>
int main(){
    printf("%d", 3 == 3.0);
return 0;
}
```

Résultat

```
1
Process returned 0 (0x0)   execution time : 0.077 s
Press any key to continue.
```

Règles de priorité

Les règles de priorité sont les suivantes, en ordre croissant :

1. L'opérateur de négation **(!).**
2. Les opérateurs relationnels (**<, <=, >, >=**).
3. Les opérateurs d'égalité et d'inégalité **(==, !=).**
4. L'opérateur logique **&&.**
5. L'opérateur logique **||**.

Exemple

```c
#include <stdbool.h>
#include <stdio.h>
int main()
{
    char a = 'a';
    int b = 33;
    double c = 12.89;
    int d = 8;
    int e = -5;
    int f = 42;
    int r;
    r = ((a < b && b > 32) && (c < d + b || e == 0)) && (f > d);
    printf("La valeur logique est égale à : %d\n", r);
    return 0;
}
```

Résultat

```
La valeur logique est Úgale Ó : 0
```

4. Actions simples

Élaboré par : Ingénieure Nahla AROUA

printf()

Les sorties représentent le transfert des données du système d'information vers l'extérieur.
Par exemple : notre ordinateur lui transmet des données pour qu'elles puissent être affichées.

Dans ce contexte, la fonction **printf()** permet d’afficher une chaine de caractères dans le console.

Affichage

En une seule ligne :

```c
#include<stdio.h>
int main(){
    printf("Bonjour tout le monde! "
           "Bonjour");
    printf("Bonjour tout le monde! ");
return 0;
}
```

Résultat

```
Bonjour tout le monde! BonjourBonjour tout le monde!
Process returned 0 (0x0)   execution time : 0.088 s
Press any key to continue.
```

En plusieurs lignes utilisant le retour à la ligne :

```c
#include<stdio.h>
int main(){
    printf("Bonjour tout le monde! \n"
           "Bonjour !");
return 0;
}
```

Résultat

```
Bonjour tout le monde!
Bonjour !
Process returned 0 (0x0)   execution time : 0.074 s
Press any key to continue.
```

Caractères spéciaux

Définition : Caractère spécial est un caractère graphique autre qu'une lettre, un chiffre, et normalement autre qu'un idéogramme (un symbole représente un mot ou une idée).

Caractère	Définition
\n	Retour à la ligne
\r	Retour chariot
\f	Saut de page
\t	Tabulation horizontale
\v	Tabulation verticale
\\	Le symbole \
\”	Le symbole “
\a	Carctère d’appel
\b	Espacement arrière

Formats

Un **format** commence toujours par le caractère **%** et est suivi par une ou plusieurs lettres qui spécifient le type de données à afficher.

Cette suite de lettres est appelée un **indicateur de conversion**. Il permet d'indiquer à la fonction printf le type de variable à insérer.

Type	Indicateur de conversion
char	c
signed char	hhd
unsigned char	hhu, hhx ou hho
int	d ou i
unsigned int	u, x, o ou X
long	ld ou li
long long	lld ou lli
unsigned long	lu, lx ou lo
unsigned long long	llu, llx, ou llo

Type	Indicateur de conversion
long double	Lf, Le (ou LE) ou Lg (ou LG)
double	lf, e (ou E) ou g (ou G)
float	f, e (ou E) ou g (ou G)
short	hd ou hi
unsigned short	hu, hx ou ho

Exemple

```c
#include <stdio.h>
int main()
{
    char c = 'c';
    char ch = 10;
    unsigned short s = 20;
    int a = 30;
    long l = 40;
    float f = 50.;
    double d = 60.0;
    long double g = 70.0;

    printf("c = %c\n", c);
    printf("Taille de 'c' (char) : %zu octets\n", sizeof(c));
    printf("ch = %d\n", ch);
    printf("s = %u\n", s);
    printf("s = %o\n", s);
    printf("s = %x\n", s);
    printf("a = %d\n", a);
    printf("Taille de 'a' (int) : %zu octets\n", sizeof(a));
    printf("l = %li\n", l);
    printf("f = %f\n", f);
    printf("d = %e\n", d);
    printf("Taille de 'd' (double) : %zu octets\n", sizeof(d));
    printf("g = %Lg\n", g);
    return 0;
}
```

Exemple

Résultat

```
c = c
Taille de 'c' (char) : 1 octets
ch = 10
s = 20
s = 24
s = 14
a = 30
Taille de 'a' (int) : 4 octets
l = 40
f = 50.000000
d = 6.000000e+01
Taille de 'd' (double) : 8 octets
g = 4.50092e-312
```

Précision des nombres flottants

```c
#include <stdio.h>
float a;
int main()
{
    a = 42.237;
    printf("a = %.2f\n", a);
    return 0;
}
```

Résultat

```
a = 42.24
```

scanf()

- Les entrées permettent de recevoir une donnée en provenance de certains périphériques.
- **Par exemple,** notre clavier va transmettre des informations sur les touches enfoncées au processeur : notre clavier est donc une entrée.

- La fonction **scanf()** a besoin de connaître l'emplacement en mémoire de nos variables afin de les modifier.

- Afin d'effectuer cette opération, nous utilisons le symbole & (qui est en fait un opérateur que nous verrons en détail plus tard).

scanf()

```c
#include <stdio.h>

int main() {

    int cote;
    printf("Entrer le coté d'un carré (en cm) : ");

    scanf("%d", &cote);

    float aire = cote * cote;

    printf("L'aire du cercle est de %f cm².\n", aire);
    return 0;
}
```

Résultat

```
Entrer le cotÚ d'un carrÚ (en cm) : 2
L'aire du cercle est de 4.000000 cm.
```

Exercice applicatif

- Écrire un programme en C effectuant les opérations arithmétiques de base (somme, produit, différence) sur deux nombres entiers a et b fournis en entrée.

```c
#include <stdio.h>
int a,b,s,d,p;
int main()
{
    printf("Entrer un nombre entier : ");
    scanf("%d", &a);
    printf("Entrer un nombre entier : ");
    scanf("%d", &b);
    s= a+b;
    d= a-b;
    p= a*b;
    printf("Somme = %d \nDifference = %d \nproduit p = %d \n",s,d,p);
    return 0;
}
```

Merci pour votre attention !